

שיעור מספר 13

דחיסה

דחיסה

- סכמת דחיסה כוללת שני מרכיבים:
 - אלגוריתם דחיסה שמקבל כקלט רצף ביטים x ומוציא כפלט רצף ביטים $C(x)$.
 - אלגוריתם שחזור שמקבל כקלט רצף ביטים y ומוציא כפלט רצף ביטים $D(y)$.
- נדרוש:
 - אם $y = C(x)$ אז $D(y) = x$ (או אולי $D(y) \approx x$).
- נשאף ש- $\text{length}(C(x)) < \text{length}(x)$. אם הדחיסה משמרת מידע (lossless) אי אפשר לדרוש זאת לכל קלט x .

קידוד Huffman

- נתון קובץ טקסט = רצף של n תווים מא"ב בגודל k .
- קידוד Huffman מנצל את התדירות השונה של התווים השונים בקובץ על מנת לחסוך בביטים.
- בקידוד unicode נזדקק ל- $16n$ ביטים.
- בקידוד עם מס' קבוע של ביטים לתו, צריך $\log k$ ביטים לתו, סה"כ $n \log k$ ביטים.
- בקידוד Huffman תווים שכיחים מקודדים בפחות ביטים מתווים נדירים.
- לשחזור קל נדרוש: לכל תו, קידודו אינו רישא של הקידוד של תו אחר.

דוגמה

- בקובץ 100,000 תווים מהא"ב {א,ב,ג,ד,ה,ו}.

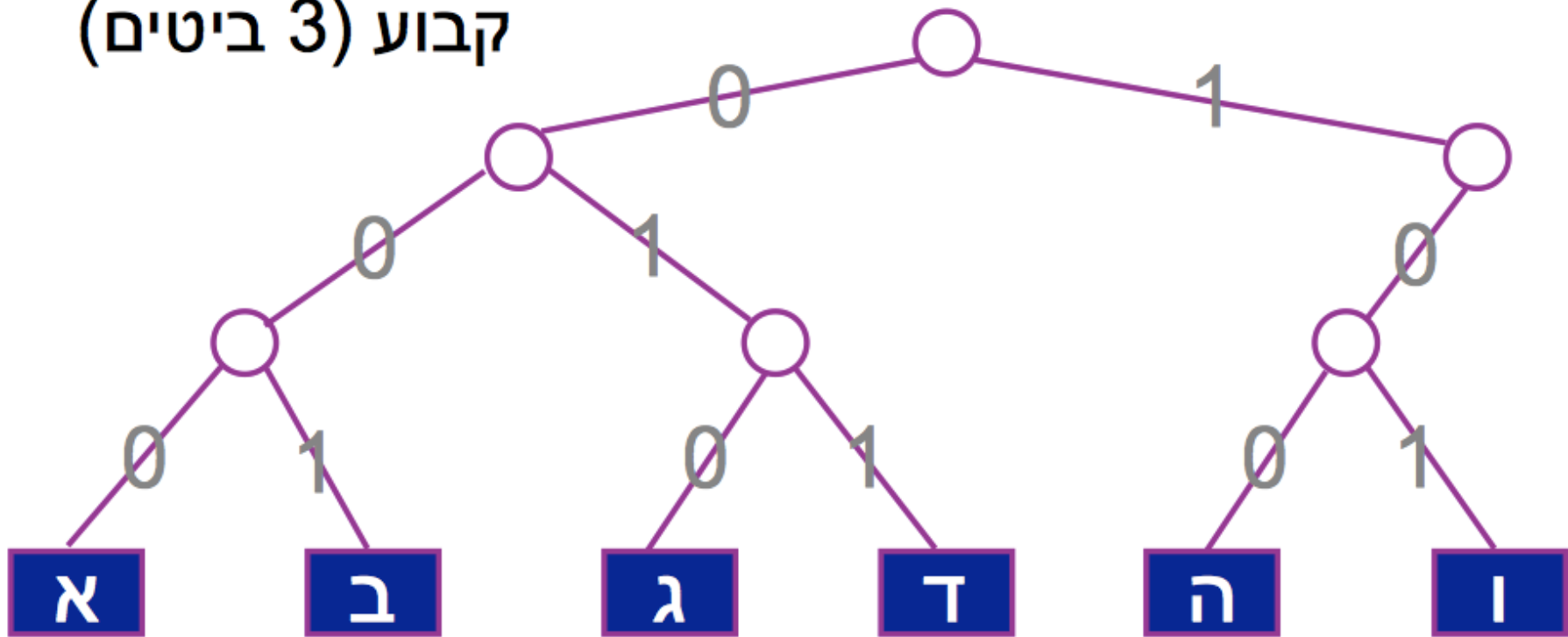
סה"כ	ו	ה	ד	ג	ב	א	
100%	5%	9%	16%	12%	13%	45%	שכיחות
300,000	101	100	011	010	001	000	אורך קבוע
224,000	1100	1101	111	100	101	0	אורך משתנה

ייצוג הקידוד

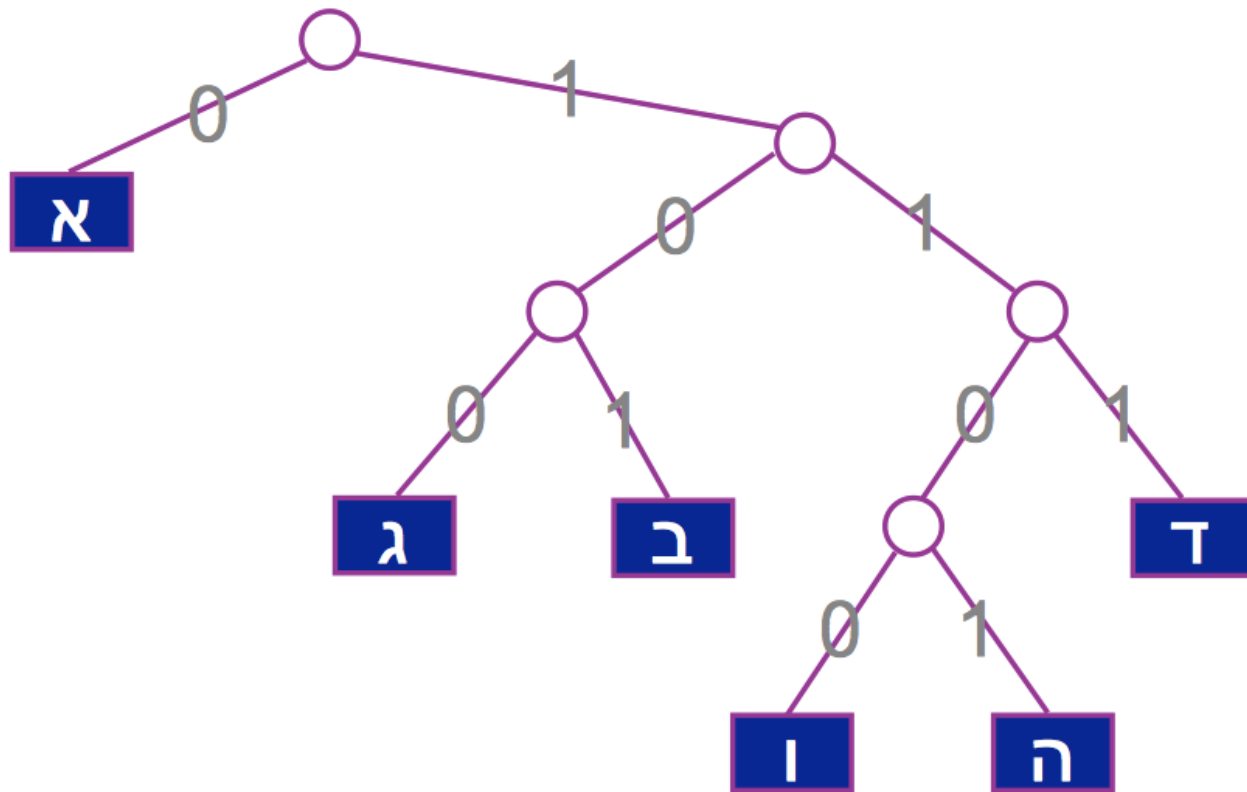
- נייצג את הקידוד על ידי עץ בינארי.
- העלים יסומנו על ידי התווים.
- מצביע לילד שמאלי יסומן ב-0.
- מצביע לילד ימני יסומן ב-1.

דוגמה

העץ המתאים לקוד
קבוע (3 ביטים)



דוגמה



העץ המתאים לקוד אופטימלי

קצת קוד

```
>>> def count_chars(text):  
    # return a dictionary of char: char-count-in-text pairs  
    dic = {}  
    for c in text:  
        if c in dic:  
            dic[c] += 1  
        else:  
            dic[c] = 1  
    return dic  
  
>>> count_chars("qwertyuiopoiuytrewqwertytrewer")  
{ 'y': 3, 'u': 2, 'i': 2, 'p': 1, 'q': 2, 'r': 6, 'o': 2, 't': 5, 'e': 5, 'w': 4 }  
>>>
```

Ln: 56 Col: 4

איך בונים קידוד Huffman?

- באינדוקציה:
- מתחילים מרשימת העלים (התווים). לכל עלה יש תדירות.
- כל עוד יש ברשימה לפחות שני צמתים:
 - שולפים מהרשימה את x ו- y - שני הצמתים עם התדירות הקטנה ביותר.
 - מייצרים צומת חדש z שהוא ההורה של x ו- y ותדירותו סכום התדירויות של x ו- y .
 - מוסיפים את z לרשימה (במקום x ו- y).

עוד קצת קוד

```
>>> def build_Huffman_code_tree(char_count):
    # char_count is a dictionary of char: char-count pairs
    nodes = []
    # create a binary tree node for each character
    for x in char_count:
        nodes.append(BinaryNode([x, char_count[x]]))
    # sort the nodes by counts
    nodes.sort(key=lambda n: n.data[1])
    # build tree - repeatedly join 2 nodes w/smallest counts
    while len(nodes) > 1:
        # extract the two nodes with smallest counts
        node1 = nodes.pop(0) #least frequent char
        node2 = nodes.pop(0) # second-least frequent char
        chars = node1.data[0] + node2.data[0] # new node
        count = node1.data[1] + node2.data[1] # its count
        # create a parent of the two extracted nodes
        parent = BinaryNode([chars, count], node1, node2)
        # find the index where to insert the new node
        counts = [n.data[1] for n in nodes]
        i = bisect.bisect(counts, count)
        nodes.insert(i, parent)
    return nodes[0]
```

סיבוכיות

- קצת מעבר לרמת הקורס ...
- עבור א"ב בגודל n :
 - המיון הראשוני לוקח $O(n \log n)$ פעולות.
 - מבצעים $2(n-1)$ הוצאות מינימום ו- $(n-1)$ הכנסות של צומת חדש. אפשר לבצע כל פעולה כזו בזמן $O(\log n)$.
- רימינו קצת, כי אנחנו בכל איטרציה מעתיקים את התדירויות לתוך `counts`. זה מעלה את הסיבוכיות ל- $O(n^2)$. אפשר לחסוך פעולה זו.

דוגמה לקידוד הפלט

```
['e', 139] 000
['I', 4] 00100000
['A', 4] 00100001
['F', 4] 00100010
['?', 2] 001000110
['j', 2] 001000111
['w', 18] 001001
['y', 19] 001010
['V', 2] 001011000
['p', 1] 0010110010
['O', 1] 0010110011
['!', 5] 00101101
['L', 5] 00101110
['D', 5] 00101111
['n', 73] 0011
['d', 74] 0100
['a', 82] 0101
['s', 40] 01100
['i', 42] 01101
['o', 85] 0111
['"', 22] 100000
['m', 22] 100001
['C', 11] 1000100
['S', 5] 10001010
```

```
['c', 6] 10001011
['.', 24] 100011
['b', 12] 1001000
['H', 6] 10010010
['B', 6] 10010011
['T', 6] 10010100
[';', 3] 100101010
['v', 3] 100101011
['N', 3] 100101100
['W', 3] 100101101
['x', 7] 10010111
['g', 26] 100110
['u', 26] 100111
['h', 103] 1010
['t', 104] 1011
[' ', 211] 110
['f', 29] 111000
['R', 7] 11100100
['P', 1] 1110010100
['-', 2] 1110010101
[':', 4] 111001011
['.', 8] 11100110
['k', 8] 11100111
['\n', 60] 11101
['l', 60] 11110
['r', 68] 11111
```

בניית טבלת הקידוד

```
>>> def build_Huffman_code_table(tree, prefix = ""):
    # builds a table of char: code pairs, given a code tree
    # for demo only the code values are character sequences
    table = {}
    if tree.is_leaf(): # the node represents a character
        table[tree.data[0]] = prefix
    else:
        if tree.left != None:
            table.update(build_Huffman_code_table(tree.left, prefix + '0'))
        if tree.right != None:
            table.update(build_Huffman_code_table(tree.right, prefix + '1'))
    return table

>>> build_Huffman_code_table(tree)
{' ': '100011', '-': '1110010101', '.': '11100110', ' ': '110', '!': '00101101', '"': '10000
0', ':': '111001011', ';': '100101010', '?': '001000110', '\n': '11101', 'h': '1010', 'i': '
01101', 'j': '001000111', 'k': '11100111', 'l': '11110', 'm': '100001', 'n': '0011', 'o': '0
111', 'a': '0101', 'b': '1001000', 'c': '10001011', 'd': '0100', 'e': '000', 'f': '111000',
'g': '100110', 'x': '10010111', 'y': '001010', 'p': '0010110010', 'r': '11111', 's': '01100'
, 't': '1011', 'u': '100111', 'v': '100101011', 'w': '001001', 'H': '10010010', 'I': '001000
00', 'L': '00101110', 'N': '100101100', 'O': '0010110011', 'A': '00100001', 'B': '10010011',
'C': '1000100', 'D': '00101111', 'F': '00100010', 'P': '1110010100', 'R': '11100100', 'S': '
10001010', 'T': '10010100', 'V': '001011000', 'W': '100101101'}
```

דחיסה

```
>>> def Huffman_compress(text):  
    tree = build_Huffman_code_tree(count_chars(text))  
    table = build_Huffman_code_table(tree)  
    compressed = "".join(table[c] for c in text)  
    return [tree, compressed]
```

שחזור

```
>>> def decompress_generator(file):  
    curr = file[0]  
    for b in file[1]:  
        if b == '0':  
            curr = curr.left  
        else: # b == '1'  
            curr = curr.right  
        if curr.is_leaf():  
            yield curr.data[0]  
    curr = file[0]  
  
>>> def Huffman_decompress(file):  
    return "".join(c for c in decompress_generator(file))
```

דוגמה

```
>>> decompress(compress("The Charge of the Light Brigade\nHalf a league, half a league,\nHalf a league onward,\nAll in the valley of Death,\nRode the six hundred.\n'Forward, the Light Brigade!\nCharge for the guns' he said:\nInto the valley of Death\nRode the six hundred.\n\n'Forward, the Light Brigade!'\nWas there a man dismay'd?\nNot tho' the soldiers knew\nSome one had blunder'd:\nTheir's not to make reply,\nTheir's not to reason why,\nTheir's but to do and die:\nInto the valley of Death\nRode the six hundred.\n\nCannon to right of them,\nCannon to left of them,\nCannon in front of them\nVolley'd and thunder'd;\nStorm'd at with shot and shell,\nBoldly they rode and well,\nInto the jaws of Death,\nInto the mouth of Hell\nRode the six hundred.\n\nFlash'd all their sabres bare,\nFlash'd as they turned in air\nSabring the gunners there,\nCharging an army while\nAll the world wonder'd:\nPlunged in the battery-smoke\nRight thro' the line they broke;\nCossack and Russian\nReel'd from the sabre-stroke\nShatter'd and sunder'd.\nThen they rode back, but not\nNot the six hundred.\n\nCannon to right of them,\nCannon to left of them,\nCannon behind them\nVolley'd and thunder'd;\nStorm'd at with shot and shell,\nWhile horse and hero fell,\nThey that had fought so well\nCame thro' the jaws of Death,\nBack from the mouth of Hell,\nAll that was left of them,\nLeft of six hundred.\n\nWhen can their glory fade?\nO the wild charge they made!\nAll the world wonder'd.\nHonour the charge they made!\nHonour the Light Brigade,\nNoble six hundred!")
```

```
"The Charge of the Light Brigade\nHalf a league, half a league,\nHalf a league onward,\nAll in the valley of Death,\nRode the six hundred.\n'Forward, the Light Brigade!\nCharge for the guns' he said:\nInto the valley of Death\nRode the six hundred.\n\n'Forward, the Light Brigade!'\nWas there a man dismay'd?\nNot tho' the soldiers knew\nSome one had blunder'd:\nTheir's not to make reply,\nTheir's not to reason why,\nTheir's but to do and die:\nInto the valley of Death\nRode the six hundred.\n\nCannon to right of them,\nCannon to left of them,\nCannon in front of them\nVolley'd and thunder'd;\nStorm'd at with shot and shell,\nBoldly they rode and well,\nInto the jaws of Death,\nInto the mouth of Hell\nRode the six hundred.\n\nFlash'd all their sabres bare,\nFlash'd as they turned in air\nSabring the gunners there,\nCharging an army while\nAll the world wonder'd:\nPlunged in the battery-smoke\nRight thro' the line they broke;\nCossack and Russian\nReel'd from the sabre-stroke\nShatter'd and sunder'd.\nThen they rode back, but not\nNot the six hundred.\n\nCannon to right of them,\nCannon to left of them,\nCannon behind them\nVolley'd and thunder'd;\nStorm'd at with shot and shell,\nWhile horse and hero fell,\nThey that had fought so well\nCame thro' the jaws of Death,\nBack from the mouth of Hell,\nAll that was left of them,\nLeft of six hundred.\n\nWhen can their glory fade?\nO the wild charge they made!\nAll the world wonder'd.\nHonour the charge they made!\nHonour the Light Brigade,\nNoble six hundred!"
```

```
>>>
```

קידוד Ziv-Lempel

- במקום לנצל הבדלים בתדירויות התווים, הקידוד מנצל את העובדה שטקסט (בשפה טבעית) נוטה לכלול חזרות על קטעים.

דוגמה

שלג על מפתנך

ועוד מעט אני אלך
את כרגיל תמצאי לך תחליף
ותטבעי איתו במשקה חריף.

שלג על מפתנך

וכל שמתחתיו נמוג, הולך
אל תאשימי את האקלים
אל תצאי שוב מן הכלים

גורלנו תמיד נשזר מכפור הפרידות
הפרדות, את יודעת, נסי על עצמך לשמור
נסי לא להסתלק מדעת
נסי לחשוב ולנשום
נסי לבעור בסנה החלום
נסי על עצמך לשמור

כששלג ירד על ירושלים הלבנה
רציתי שנהיה יחד
לקלוע את ידי בשערך
להביט ביופיך ולהישתאות
מה שנשמט מימינו ואבד

קידוד Ziv-Lempel

- הרעיון המרכזי בקידוד:

- אם $\text{text}[i:i+k] = \text{text}[j:j+k]$ עבור $j > i$, אז נקודד את $\text{text}[j:j+k]$ ע"י הזוג $j-i, k$. (נחפש k מירבי.)
- כמובן, אין טעם לעשות זאת אם k קטן מאוד.
- נדרוש $k \geq 3$ (בדר"כ מסתפקים ב- $k \geq 2$).
- נהוג להגביל את אורך החזרה המירבי: $k \leq 31$.
- גם אין טעם לעשות זאת אם $j-i$ מאוד גדול.
- נהוג להגביל את חלון הטקסט w שבו מחפשים חזרה בעבר. ערך טיפוסי: $w = 4095$.
- עם הערכים לעיל, קידוד קטע חוזר דורש 17 ביטים.

מימוש

- הבעייה המרכזית היא איך לאתר את החזרה המקסימלית ביעילות.
- אנחנו נחזיק מילון של השלשות שמופיעות בחלון עם רשימת המקומות שכל שלשה הופיעה.
- נשלוף מקומות מועמדים לפי השלשה ועבור כל מועמד נבדוק מה אורך החזרה.
- בקורס "מבני נתונים" תלמדו כלים אחרים לעשות זאת.

בספריות פייתון

- פייתון מספקת מימוש של gzip, גרסה של דחיסת ZL שפותחה במסגרת פרויקט GNU *
 - המודול נמצא ב-Lib/gzip.py
 - משתמש במודול zlib, ראו:
- <http://www.zlib.net>
- אלגוריתם הדחיסה: ZL+Huffman

בספריות פייתון

- פייתון מספקת מימוש של gzip, גרסה של דחיסת ZL שפותחה במסגרת פרוייקט GNU *
 - המודול נמצא ב-Lib/gzip.py
 - משתמש במודול zlib, ראו:
- <http://www.zlib.net>
- אלגוריתם הדחיסה: ZL+Huffman

* השם GNU הוא ראשי תיבות של GNU's not Unix. זהו שם הפרוייקט המרכזי של המוסד לתוכנה חופשית (Free Software Foundation).
גנו, במקור משפת קויקוי (הוטנטוטית) היא אנטילופה גדולה מאפריקה. נקראת לחילופין וילדהביסט (חיה פראית באפריקאנס).

בספריות פייתון

- פייתון מספקת מימוש של gzip, גרסה של דחיסת ZL שפותחה במסגרת פרוייקט GNU *
- המודול נמצא ב-Lib/gzip.py
- משתמש במודול zlib, ראו:
<http://www.zlib.net>
- אלגוריתם הדחיסה: ZL+Huffman



גנו

* השם GNU הוא ראשי תיבות של GNU's not Unix. זהו שם הפרוייקט המרכזי של המוסד לתוכנה חופשית (Free Software Foundation).
גנו, במקור משפת קויקוי (הוטנטוטית) היא אנטילופה גדולה מאפריקה. נקראת לחילופין וילדהביסט (חיה פראית באפריקאנס).

קוד הקידוד

```
Python 3.3.3 (v3.3.3:c3896275c0f6, Nov 16 2013, 23:39:35)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> def z177_compress(text, window = 2**12-1, rep_limit = 2**5-1, min_rep = 3):
    compressed = []
    for (pos, i, len) in z177_max_repetition(text, window, rep_limit, min_rep):
        if len < min_rep:
            compressed.append(text[pos])
        else:
            compressed.append((pos-i, len))
    return compressed
```

הפונקציה המרכזית

```
>>> def zl77_max_repetition(text, window = 2**12-1, rep_limit = 2**5-1, min_rep = 3):
    # Repeatedly find the maximum substring in text[pos-window:pos]
    # that matches the prefix of text[pos:], up to a length of rep_limit.
    # A repetition of length < min_rep is not reported.
    # If the max repetition < min_rep, return pos, 0, 0
    # otherwise, return pos, max-rep-position, max-rep-length
    dic = TuplesDic() # dictionary of past min_rep-tuples, to find repetitions quickly
    pos = 0 # current position in text
    while pos < len(text):
        if pos <= len(text) - min_rep: # we have a min_rep-tuple
            tup = text[pos:pos+min_rep]
            candid_list = dic.get(tup)
            max_rep_len = 1
            if candid_list != None: # there's a past repetition
                for i in candid_list: # find the longest past repetition
                    k = 0
                    # loop to determine length of current repetition
                    while pos+k < len(text) and text[i+k] == text[pos+k] and k <= rep_limit:
                        k += 1
                    if k > max_rep_len: # current repetition is longer
                        max_rep_len = k
                        max_rep_pos = i
                yield pos, max_rep_pos, max_rep_len
            else: # no past repetition
                yield pos, 0, 0
            for i in range(max_rep_len): # skip the encoded segment and update dic
                dic.put(text[pos+i:pos+i+min_rep], pos+i)
                if pos+i-window >= 0:
                    dic.pop(text[pos+i-window:pos+i-window+min_rep])
            pos += max_rep_len
        else: # last min_rep-1 characters
            yield pos, 0, 0
            pos += 1
```

דוגמה

```
>>> import collections
>>> z177_compress("Mister Your eyes are full of hesitation Sure makes me wonder If you know what you're looking for.
Baby I wanna keep me reputation I'm a sensation You try me once, you'll beg for more Yes Sir, I can boogie But I need
a certain song. I can boogie, Boogie woogie All night long. Yes Sir, I can boogie If you stay, you cant't go wrong. I
can boogie, Boogie woogie All night long. No Sir I don't feel very much like talking No, neither walking You wanna kn
ow if I can dance Yes Sir Already told you in the first verse And in the chorus But I will give you one more chance.
Yes Sir, I can boogie")
['M', 'i', 's', 't', 'e', 'r', 'Y', 'o', 'u', 'r', ' ', 'e', 'y', 'e', 's', ' ', 'a', 'r', 'e', ' ', 'f', 'u', 'l',
'l', ' ', ' ', 'o', 'f', ' ', 'h', 'e', 's', 'i', 't', 'a', 't', 'i', 'o', 'n', ' ', 'S', 'u', 'r', 'e', ' ', 'm', 'a', 'k', 'e',
's', ' ', 'm', 'e', ' ', 'w', 'o', 'n', 'd', 'e', 'r', ' ', 'I', 'f', ' ', 'y', 'o', 'u', ' ', 'k', 'n', 'o', 'w', ' ', 'w', 'h',
'a', 't', ' ', 'y', 'o', 'u', ' ', 'l', 'o', 'o', 'k', ' ', 'i', 'n', ' ', 't', 'h', 'e', ' ', 'f', 'i', 'r', 's', 't', ' ', 'v', 'e',
'r', 's', 'e', ' ', 'A', 'n', 'd', ' ', 'i', 'n', ' ', 't', 'h', 'e', ' ', 'c', 'h', 'o', 'r', 'u', 's', ' ', 'B', 'u', 't', ' ', 'I', ' ', 'w', 'i',
'l', 'l', ' ', 'g', 'i', 'v', 'e', ' ', 'y', 'o', 'u', ' ', 'o', 'n', 'e', ' ', 'm', 'o', 'r', 'e', ' ', 'c', 'h', 'a', 'n', 'c', 'e', ' ',
'Y', 'e', 's', ' ', 'S', 'i', 'r', ' ', 'I', ' ', 'c', 'a', 'n', ' ', 'b', 'o', 'o', 'g', 'i', 'e', ' ', 'B', 'u', 't', ' ', 'I', ' ', 'n', 'e',
'e', 'd', ' ', 'a', ' ', 'c', 'e', 'r', 't', 'a', 'i', 'n', ' ', 's', 'o', 'n', 'g', ' ', 'I', ' ', 'c', 'a', 'n', ' ', 'b', 'o', 'o', 'g', 'i',
'e', ' ', 'B', 'o', 'o', 'g', 'i', 'e', ' ', 'w', 'o', 'o', 'g', 'i', 'e', ' ', 'A', 'l', 'l', ' ', 'n', 'i', 'g', 'h', 't', ' ', 'l', 'o', 'n',
'g', ' ', 'Y', 'e', 's', ' ', 'S', 'i', 'r', ' ', 'I', ' ', 'c', 'a', 'n', ' ', 'b', 'o', 'o', 'g', 'i', 'e', ' ', 'I', 'f', ' ', 'y', 'o',
'u', ' ', 's', 't', 'a', 'y', ' ', 'y', 'o', 'u', ' ', 'c', 'a', 'n', 't', ' ', 'g', 'o', ' ', 'w', 'r', 'o', 'n', 'g', ' ', 'I', ' ', 'c',
'a', 'n', ' ', 'b', 'o', 'o', 'g', 'i', 'e', ' ', 'B', 'o', 'o', 'g', 'i', 'e', ' ', 'w', 'o', 'o', 'g', 'i', 'e', ' ', 'A', 'l', 'l', ' ', 'n',
i', 'g', 'h', 't', ' ', 'l', 'o', 'n', 'g', ' ', 'N', 'o', ' ', 'S', 'i', 'r', ' ', 'I', ' ', 'd', 'o', 'n', 't', ' ', 'f', 'e', 'e', 'l', ' ',
v', 'e', 'r', 'y', ' ', 'm', 'u', 'c', 'h', ' ', 'l', 'i', 'k', 'e', ' ', 't', 'a', 'l', 'k', 'i', 'n', 'g', ' ', 'N', 'o', ' ', 'n', 'e', 'i',
t', 'h', 'e', 'r', ' ', 'w', 'a', 'l', 'k', 'i', 'n', 'g', ' ', 'Y', 'o', 'u', ' ', 'w', 'a', 'n', 'n', 'a', ' ', 'k', 'n', 'o', 'w', ' ', 'i',
f', ' ', 'I', ' ', 'c', 'a', 'n', ' ', 'd', 'a', 'n', 'c', 'e', ' ', 'Y', 'e', 's', ' ', 'S', 'i', 'r', ' ', 'A', 'l', 'r', 'e', 'a', 'd', 'y', ' ',
t', 'o', 'l', 'd', ' ', 'y', 'o', 'u', ' ', 'i', 'n', ' ', 't', 'h', 'e', ' ', 'f', 'i', 'r', 's', 't', ' ', 'v', 'e', 'r', 's', 'e', ' ', 'A',
n', 'd', ' ', 'i', 'n', ' ', 't', 'h', 'e', ' ', 'c', 'h', 'o', 'r', 'u', 's', ' ', 'B', 'u', 't', ' ', 'I', ' ', 'w', 'i', 'l', 'l', ' ', 'g', 'i',
v', 'e', ' ', 'y', 'o', 'u', ' ', 'o', 'n', 'e', ' ', 'm', 'o', 'r', 'e', ' ', 'c', 'h', 'a', 'n', 'c', 'e', ' ', 'Y', 'e', 's', ' ', 'S', 'i',
'r', ' ', 'I', ' ', 'c', 'a', 'n', ' ', 'b', 'o', 'o', 'g', 'i', 'e', ' ']
```